

★ Member-only story

Gradient Boosting +

Gradient Descent +

Machine Learning +

Math ✓

Data Science ✓

# The Accidental Discovery That Shaped Every Prediction You Make as a Data Scientist



Dr Swarnendu AI

Following ▾

12 min read · 13 hours ago



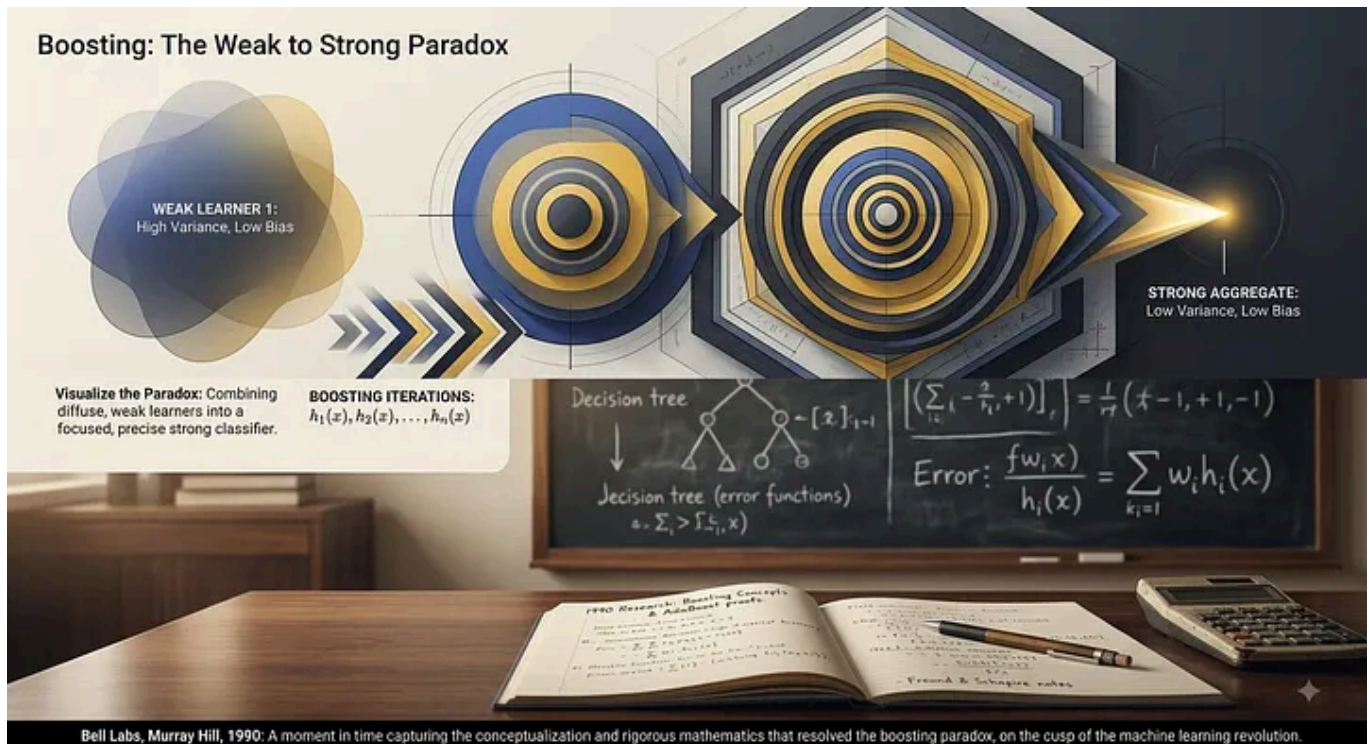
86



1



*How One Theoretical Question Led to the Algorithm That Powers Modern Machine Learning*



You're sitting in a classroom at Bell Labs in 1990. The year carries no particular weight — no one is aware yet that the next five minutes will reshape how machines learn.

Robert Schapire, working through the foundations of learning theory, reaches a conclusion that stands in seeming contradiction to everything the field has built:

“If you can construct a learner that performs only marginally better than random chance, a formal procedure exists to combine many such learners into a predictor of arbitrary accuracy.”

The statement arrives as an inversion of intuition. In the prevailing understanding, weak signals remain weak. Noise combined remains noise.

The mathematics, however, contains no such mercy for conventional thinking.

Schapire possesses not a conjecture but a proof.

Five years would pass before anyone successfully implemented it.

When implementation came, it arrived not as the fulfillment of an ambition, but as a byproduct of trying to solve a different problem entirely.

## The Theorem That Reversed the Intuition

The preceding decade had posed a question of theoretical elegance with practical consequences.

In 1984, Leslie Valiant introduced **PAC Learning** — Probably Approximately Correct learning — a framework for reasoning about when learning from data becomes possible. Within this framework emerged a fundamental inquiry: *Can weak learners be combined through some formal mechanism to produce strong learning?*

The conventional answer was no. Weak signals, when aggregated, yield only weak signal. The mathematics of probability seems to forbid otherwise.

In 1990, Schapire proved otherwise.

His result was rigorous and complete: if a procedure exists for constructing a learner that exceeds random guessing — even by a margin as slender as one percent — then a formal mechanism exists to combine these learners into a predictor achieving arbitrarily high accuracy.

The mathematics was impeccable. The practical implementation, however, required sample sizes that existed nowhere in the physical world. The theorem was theoretically pure and practically impossible.

Schapire published it. The insight entered the literature. The field moved forward in other directions.

Five years later, in 1995, Schapire and Yoav Freund converged on a shared question: the gap between Schapire's theoretical guarantee and the practical world remained vast. The inquiry shifted from *whether* weak learners could be combined to *how* they might be combined efficiently enough to matter.

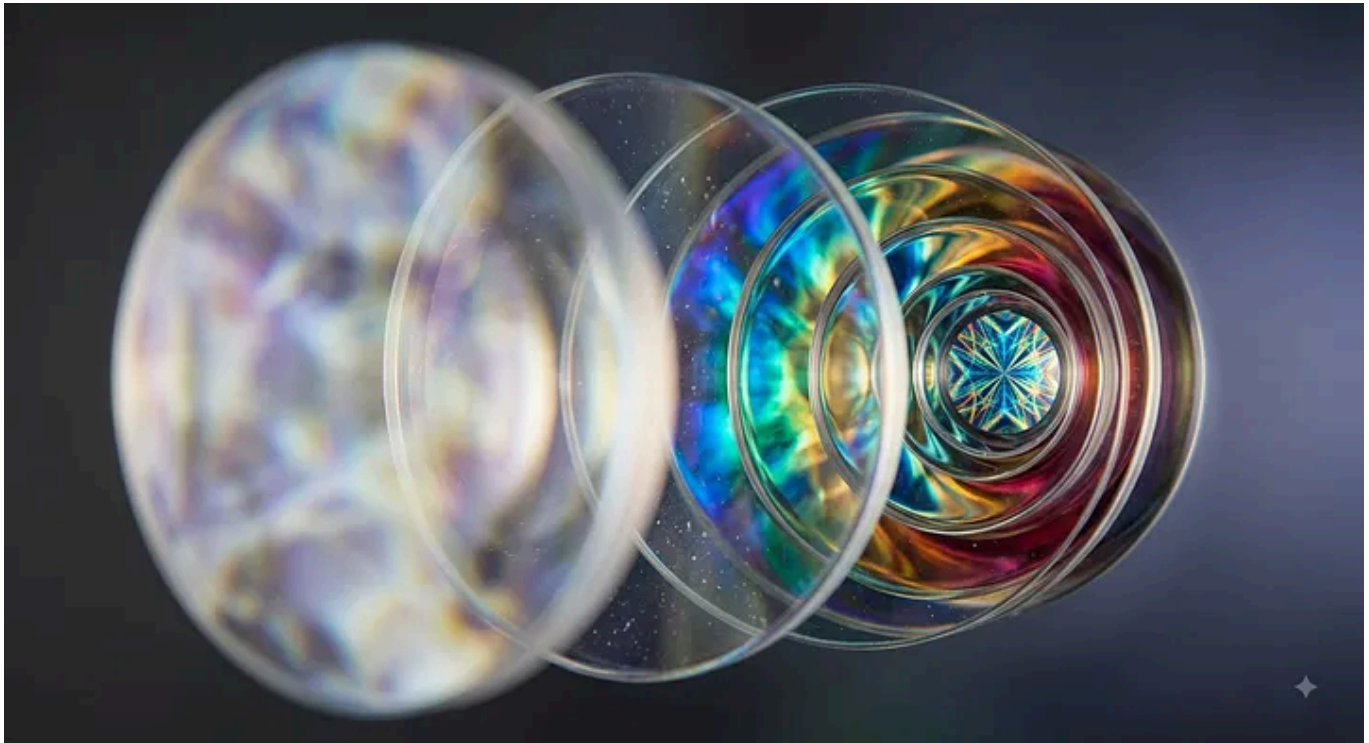
The solution Freund developed carried the quality of discovered rather than invented. It was simple enough to describe in five steps:

1. Train a weak learner on your data
2. Identify the instances it misclassified; weight these more heavily
3. Train a second weak learner on this reweighted data
4. Repeat this process
5. Combine predictions through aggregation

The algorithm was christened **AdaBoost** — Adaptive Boosting. The name captured its essential character: the weighting adapted after each iteration, focusing learning on the examples that remained difficult.

When tested, the results were unexpected even to its designers. AdaBoost did not merely work. It outperformed all existing methods on digit recognition, character recognition, spam detection, and classification

problems with margins that seemed to violate some unspoken rule of machine learning competition.



What renders this history remarkable is not that they built an algorithm — it is that they built it while pursuing a theoretical completion, not practical victory. The competitive dominance became an accessory to their central mission.

By 1997, when the work appeared in publication, the response was immediate. Within two years, AdaBoost had become the reference method in the field. Competitions were won with it. Production systems were built around it. What had been theoretical became indispensable.

Schapire and Freund received the Gödel Prize — mathematics' highest recognition.

Yet neither they nor the field possessed a complete explanation for why their algorithm performed so extraordinarily well. This gap between empirical dominance and theoretical understanding would occupy the field's attention for years.

## **The Mechanism Revealed**

The intuitive model of AdaBoost carries immediate appeal: an instructor identifies students who struggle, devotes focused attention to those difficulties, then reassesses. With each cycle, the previously difficult problems yield to mastery. The metaphor captures an important truth.

Yet it describes the surface, not the engine.

For several years following publication, the learning theory community developed multiple interpretations. Various frameworks offered partial insights. The complete picture — the mathematical mechanism unifying Schapire's theoretical guarantee with Freund's empirical dominance — required a conceptual reorientation.

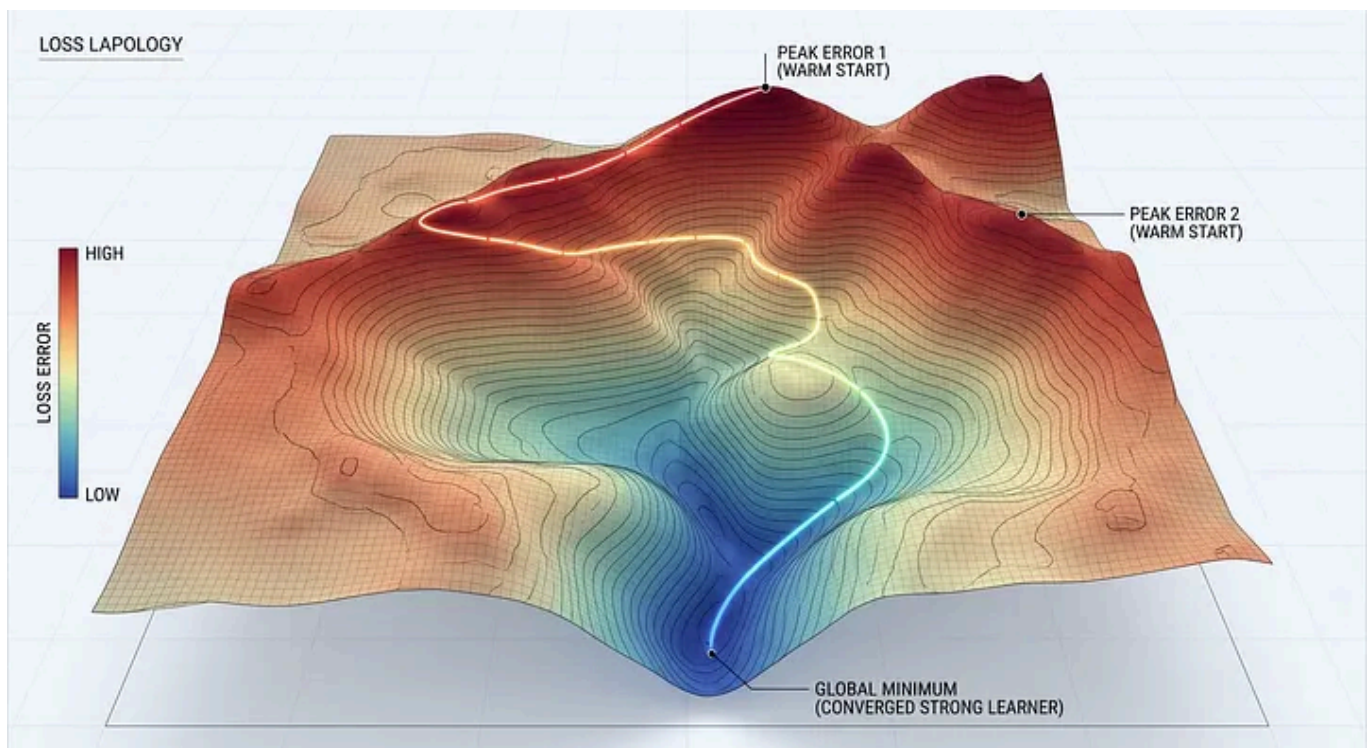
Jerome Friedman, conducting extensive theoretical work through the late 1990s and early 2000s, made a crucial recognition: boosting was not fundamentally about example reweighting. It was gradient descent performed in the space of ensemble classifiers.

**The actual mechanism:**



When you reweight misclassified instances, you are implicitly computing the direction in which your ensemble's loss decreases most steeply. Each new weak learner constitutes a step in this direction. The ensemble becomes not a collection of independent models, but a single evolving entity, iteratively updated and moving through parameter space toward lower loss.

The analogy is precise: imagine descending a mountain in complete darkness. You cannot perceive the summit. You can only determine which direction slopes most steeply downward. You move in that direction. Boosting operates identically, except the mountain is your loss landscape and each step is a weak learner.



Friedman's formalization in the early 2000s revealed something of deeper significance: boosting was not an isolated technique. It was a manifestation

of a broader principle.

## The Generalization That Revealed the Pattern

In 2001, Jerome Friedman published “Greedy Function Approximation: A Gradient Boosting Machine.” The title possessed no glamour. The presentation was rigorous and dense. Yet the work contained a unifying insight of considerable power.

Friedman demonstrated that AdaBoost represented a specific instantiation of a more general framework: **gradient boosting**.

The core principle is elegantly simple:

You possess a loss function measuring prediction error. You maintain an ensemble. At each iteration, you must select which model to add. The optimal choice is the model that most steeply reduces loss — the one pointing most directly downhill on your loss surface.

This is gradient boosting in its essence.

Viewed through this lens, AdaBoost is revealed as gradient boosting optimized for exponential loss. One could equivalently construct gradient boosting with squared error loss, or log-loss, or any differentiable loss function. The framework admits this flexibility.

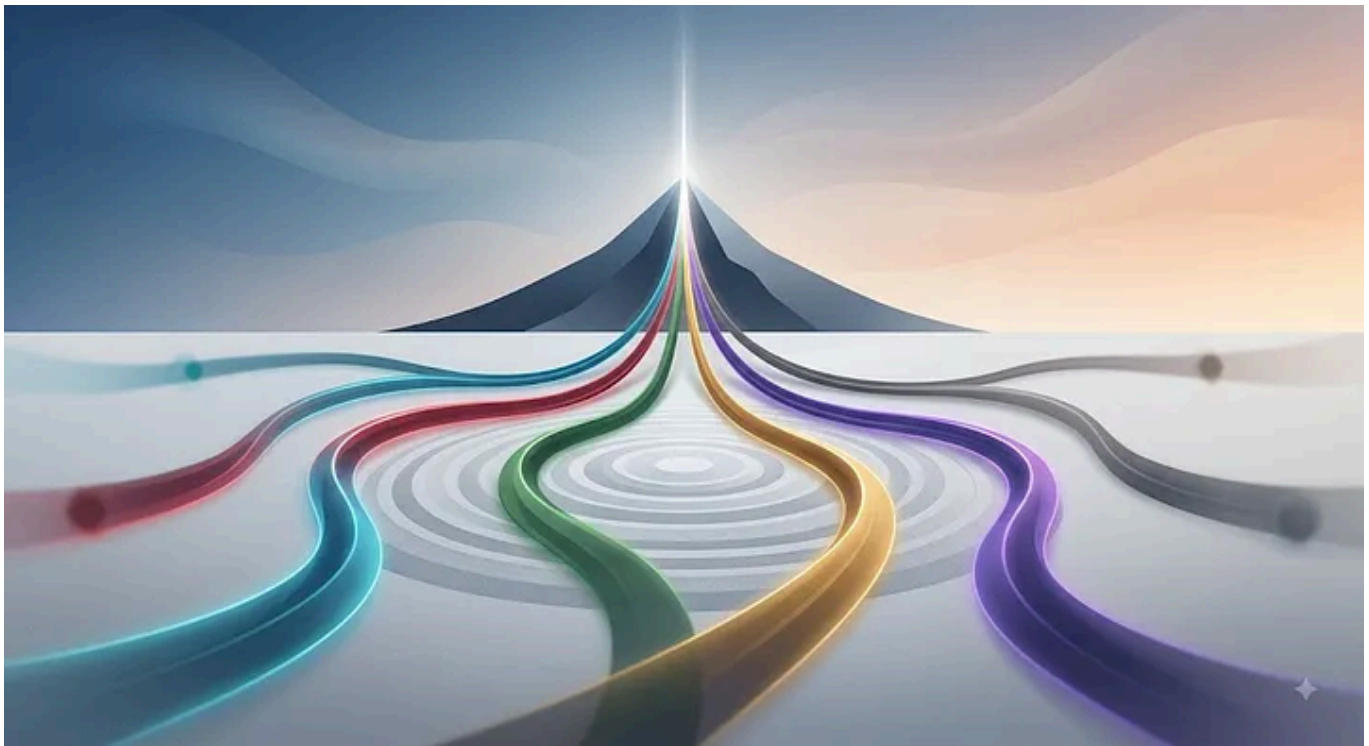
The implications extended across the entire field:

- Regression and classification, once distinct problems, became unified under a single algorithmic framework



- Any metric of interest could be optimized by appropriate loss function definition
- The same approach applied equally to decision trees, linear models, or neural networks as base learners

Adoption, however, proceeded cautiously. Gradient boosting with trees required substantial computation. The algorithms were inherently sequential — parallelization offered limited benefit. Complexity of implementation discouraged adoption.



For a decade following this unification, AdaBoost remained the dominant practical choice. Competition platforms selected it. Production systems relied on it. It was the reference method.

Then the conditions surrounding the problem changed.

## The Bottleneck

By the early 2010s, three things happened simultaneously:

1. Data sizes exploded (millions → billions of rows)
2. Competitions got serious (Kaggle launched in 2010)
3. Tree-based methods were showing they could outperform deep learning on structured data

But gradient boosting with trees was *slow*. Training a gradient boosted tree ensemble on a million rows took hours. On a billion rows, it wasn't feasible.

The algorithm was theoretically perfect and practically unusable.

Kaggle competitors started complaining. Teams in industry were stuck. The gradient boosting framework was the best thing they had, but it didn't scale.

This is where Chen Tianqi and Carlos Guestrin entered.

## The Moment Everything Accelerated

In 2016, Chen and Guestrin released **XGBoost** — eXtreme Gradient Boosting.

They didn't invent a new algorithm. They didn't prove a new theorem. They did something harder: they made the existing algorithm *fast*.

Here's what they did:

- 1. Engineered the tree-splitting algorithm.** Standard gradient boosting searches for the best split naively. It checks every feature, every threshold. XGBoost used approximation: histogram-based splitting. Group the values, approximate within groups, find the best split roughly instead of exactly. It's 10–100x faster and loses almost nothing.
- 2. Added regularization properly.** Gradient boosting can overfit. XGBoost built in L1 and L2 regularization as first-class citizens, not afterthoughts.
- 3. Handled sparse data.** Real-world data is sparse (lots of zeros, missing values). XGBoost was built for it from the ground up.
- 4. Parallelized what could be parallelized.** Not the sequential boosting part — that's inherently sequential — but the tree-growing part within each round.
- 5. Added column and row subsampling.** Stochastic gradient boosting ideas, but implemented efficiently.

The result: a machine learning algorithm that was theoretically grounded, practically fast, and absurdly effective.

When XGBoost was released, Kaggle winners started using it. Every competition between 2016 and 2022 that had structured data was won by XGBoost or LightGBM or CatBoost (competing implementations of the same idea).

It wasn't hype. It was dominance.

## Why Modern ML Swears By It

Today, if you work in industry, you've probably used gradient boosting. Directly or indirectly.

Here's why it won:

**Effectiveness on structured data:** Gradient boosting with trees is, empirically, the best general-purpose algorithm for tabular data. Neural networks are incredible for unstructured data (images, text, audio), but on a CSV file with 50 columns, a boosted tree beats a neural network 95% of the time.

**Interpretability:** You can see what XGBoost learned. Feature importance. SHAP values. You can explain each prediction. Try that with a neural network.

**Sample efficiency:** Gradient boosting works with smaller datasets. You need thousands of rows, not millions. This matters in real product teams where you don't have massive datasets.

**Robustness:** XGBoost doesn't care if you have missing values, categorical features, or features on different scales. It just works. Neural networks require preprocessing that borders on shamanic ritual.

**Speed:** Modern implementations are *fast*. XGBoost, LightGBM, CatBoost can train on millions of rows in seconds.

**Mathematical foundation:** It's not magic. It's a direct consequence of gradient descent in ensemble space. You can understand *why* it works, not just that it does.

This combination — theoretical purity, practical speed, empirical dominance, human interpretability — is rare in machine learning. It's why gradient boosting is the algorithm that *doesn't die*.

## The Math, Finally Explained

Let me show you what's actually happening.

You start with a weak learner. Let's say a decision tree with depth 1 (a stump — just one split).

You train it on your data:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import make_regression

# Generate simple data
X, y = make_regression(n_samples=100, n_features=5, noise=10,
random_state=42)

# First weak learner
learner_1 = DecisionTreeRegressor(max_depth=1)
learner_1.fit(X, y)
pred_1 = learner_1.predict(X)

# Residuals: what it got wrong
residuals_1 = y - pred_1

print(f"First learner MSE: {np.mean(residuals_1**2):.4f}")
```

The first learner gets some predictions right, some wrong. The *residuals* — the differences between true and predicted — tell you where it failed.

Now, instead of retraining on the original data, you train the next learner on the **residuals**:

```
# Second weak learner: fit to residuals
learner_2 = DecisionTreeRegressor(max_depth=1)
learner_2.fit(X, residuals_1)
pred_2 = learner_2.predict(X)
```

```
# New residuals: what both learners together got wrong
residuals_2 = residuals_1 - pred_2

print(f"After 2 learners MSE: {np.mean(residuals_2**2):.4f}")
```

Wait. You trained on the residuals from the first model. That means the second model is learning what the first model *missed*.

Do this again:

```
# Third weak learner: fit to residuals of residuals
learner_3 = DecisionTreeRegressor(max_depth=1)
learner_3.fit(X, residuals_2)
pred_3 = learner_3.predict(X)
```

```
residuals_3 = residuals_2 - pred_3

print(f"After 3 learners MSE: {np.mean(residuals_3**2):.4f}")
```

And again. And again. Each new learner fixes what the previous ones got wrong.

Your final prediction is:

```
# Final ensemble prediction
final_pred = pred_1 + pred_2 + pred_3 + ...
```

```
# Or more generally:
y_boosted = sum(learner.predict(X) for learner in learners)
```

You're literally summing the predictions. Each weak learner adds a little bit to the total.

But here's the magic: each new learner is added with a *learning rate*. Instead of adding the full second prediction, you add  $\eta$  (eta, typically 0.1) times the second prediction:

```
learning_rate = 0.1
y_boosted = (
    pred_1 +
    learning_rate * pred_2 +
    learning_rate * pred_3 +
    ...
)
```

Open in app ↗



Medium



Search



Write



Now, this is boosting in regression. Classification is similar, but with a different loss function (log-loss instead of squared error).



Here's the thing that took Friedman years to formalize: **this process is equivalent to computing the gradient of your loss function and moving in the direction of steepest descent in ensemble space.**

Each new learner is a direction. The learning rate is step size. The sequence of models is your path down the loss mountain.

That's it. That's why it works.

## Code: Building Your Own Gradient Boosting (Simplified)

Let me show you a minimal implementation so you see the mechanism clearly:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

class SimpleGradientBoosting:
    def __init__(self, n_estimators=100, learning_rate=0.1,
max_depth=1):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.models = []
        self.init_pred = None

    def fit(self, X, y):
        # Start with mean prediction
        self.init_pred = np.mean(y)
        current_pred = np.full_like(y, self.init_pred,
dtype=np.float64)

        for i in range(self.n_estimators):
            # Residuals: what we got wrong
            residuals = y - current_pred

            # Fit weak learner to residuals
```

```

        model = DecisionTreeRegressor(
            max_depth=self.max_depth,
            random_state=42
        )
        model.fit(X, residuals)

        # Update prediction
        update = self.learning_rate * model.predict(X)
        current_pred += update

        # Store model
        self.models.append(model)

        # Print progress
        if (i + 1) % 20 == 0:
            mse = mean_squared_error(y, current_pred)
            print(f"Iteration {i+1}: MSE = {mse:.4f}")

    def predict(self, X):
        pred = np.full(len(X), self.init_pred, dtype=np.float64)
        for model in self.models:
            pred += self.learning_rate * model.predict(X)
        return pred

# Test it
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=200, n_features=10, noise=15,
                      random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train
gb = SimpleGradientBoosting(n_estimators=100, learning_rate=0.1,
                           max_depth=1)
gb.fit(X_train, y_train)

# Evaluate
train_pred = gb.predict(X_train)
test_pred = gb.predict(X_test)

print(f"\nFinal Train MSE: {mean_squared_error(y_train,
train_pred):.4f}")
print(f"Final Test MSE: {mean_squared_error(y_test, test_pred):.4f}")

```

Run this. You'll see the MSE drop with each iteration. That's boosting.

## Now compare it to XGBoost:

```
from xgboost import XGBRegressor

xgb = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=1)
xgb.fit(X_train, y_train)

xgb_train = xgb.predict(X_train)
xgb_test = xgb.predict(X_test)

print(f"XGBoost Train MSE: {mean_squared_error(y_train,
xgb_train):.4f}")
print(f"XGBoost Test MSE: {mean_squared_error(y_test,
xgb_test):.4f}")
```

XGBoost will be faster and probably more accurate (it has better regularization). But the mechanism is identical.

## The Timeline of Accidents

Here's what's remarkable about this story: it's a sequence of accidents.

**1990:** Schapire proves weak learners can be combined (solving a 6-year-old theoretical question). The proof is impractical.

**1995:** Freund and Schapire, trying to make the theorem practical, accidentally invent the most powerful algorithm anyone's seen.

**1997:** AdaBoost is published. Everyone thinks it's revolutionary. Nobody knows *why* it works.

**2000:** Friedman generalizes it mathematically and realizes it's gradient descent. This wasn't obvious. It was a genuine insight.

**2010s:** Everyone using boosting. It's slow on big data. Deemed a limitation of the approach.

**2016:** Chen and Guestrin optimize the implementation (not the algorithm). Suddenly it's usable on any dataset.

Each step wasn't someone saying "I'm going to revolutionize machine learning." Each step was someone solving the immediate problem in front of them, and the revolution happened as a side effect.

## **Even Today I find it relevant**

Here's the plot twist.

In 2024, deep learning is still eating the world. Transformers are used for everything. LLMs are in production. Everyone talks about neural networks.

And yet.

In industry, 80% of production ML models are still tree-based. They're boosted trees. XGBoost. LightGBM. CatBoost.

Why? Because:

- They work better on structured data
- They train faster

- They're more interpretable
- They don't require GPU infrastructure
- They're stable and don't have weird failure modes

A researcher might publish a paper using a fancy neural architecture. A company using that result would probably port it to gradient boosting and get better results.

This is the unsexy truth: the most important algorithm in modern machine learning is one that was discovered by accident while solving an old theoretical problem. It's mathematically grounded in gradient descent. It works consistently. And almost nobody outside industry knows or cares about it.

The reason Kaggle winners have been dominated by boosting since 2016 isn't because of hype. It's because of parity.

Friedman's gradient boosting framework is, structurally, equivalent to what you're doing when you train a neural network. You're moving down a loss gradient. You're doing gradient descent. The implementation is different, the intuition is the same.

But boosting does it with much less data and no GPU.

## The Lesson

This history teaches something important about how science and engineering actually work:

You rarely set out to build something great. You set out to solve the problem in front of you.

Schapire was trying to answer a theoretical question. Freund was trying to make it practical. Friedman was trying to understand why it worked. Chen was trying to make it fast.

Each step was local. Each step was driven by immediate necessity.

And together, they built something that's run billions of predictions in production, worth hundreds of millions of dollars, and saved countless hours of engineering work.

That's not grand strategy. That's not visionary leadership. That's craftsmanship.

That's what happened with boosting.

And it's still the best algorithm you've probably never heard of.

## References & Further Reading

- Schapire, R. E. (1990). "The Strength of Weak Learnability." *Machine Learning*, 5(2), 197–227.
- Freund, Y., & Schapire, R. E. (1997). "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting." *Journal of Computer and System Sciences*, 55(1), 119–139.

- Friedman, J. H. (2001). “Greedy function approximation: A gradient boosting machine.” *Annals of Statistics*, 29(5), 1189–1232.
- Chen, T., & Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Gradient Boosting +

Gradient Descent +

Machine Learning +

Math ✓

Data Science ✓

**Written by Dr Swarnendu AI**

Following ▾

1.8K followers · 1K following

AI Learner and Ph.D.(in not so famous domain). I demystify AI using First Principles, Causal ML, and the beauty of classical math. Let's make machines reason.





